

```
1 #这里很多矩阵运算和python语法还不熟悉。。。。。
2 #以后还是得亲自手算一下小样例才行
3
4
5 #=====本篇带领我们手动进行backward，对梯度更新理解更深刻=====
6 # (1) 第一个总结：
7 #   1.如果前向传播中，从A得到B的过程，需要广播机制，也就是A需要复制自身，来对齐B：
8 #       那么后续backward从B求A的时候，因为同一个A的元素经过多次复制，运算得到B，也就是一个A的元素是多个B元素的父元素
9 #       那么backward的时候就需要沿着B的某一个维度进行sum，来累加正确得到被多次使用的A元素的grad
10
11 #   2.如果前向传播中，从A得到B的过程，需要和机制，也就是一个B元素，是多个A元素的子元素
12 #       比如B_1 = A_1 + A_2 + A_3 + ... + A_n
13 #       那么backward的时候，就需要广播机制。复制多个B1，然后分别将对B1的grad，直接传递到A_n上。
14 #       因为B_1对这些A的grad都是1，所以直接将dL / dB1传就行了
15 import torch
16 import torch.nn.functional as F
17 import matplotlib.pyplot as plt # for making figures
18 #=====初始化数据集和context=====
19 # read in all the words
20 words = open('Neural Network_Zero to hero/makemore/names.txt', 'r').read().splitlines()
21
22 # build the vocabulary of characters and mappings to/from integers
23 chars = sorted(list(set(''.join(words))))
24 stoi = {s:i+1 for i,s in enumerate(chars)}
25 stoi['.'] = 0
26 itos = {i:s for s,i in stoi.items()}
27 vocab_size = len(itos)
28 print(itos)
29 print(vocab_size)
30
31 # build the dataset
32 block_size = 3 # context length: how many characters do we take to predict the next one?
33
34 def build_dataset(words):
35     X, Y = [], []
36
37     for w in words:
```

```
38     context = [0] * block_size
39     for ch in w + '.':
40         ix = stoi[ch]
41         X.append(context)
42         Y.append(ix)
43         context = context[1:] + [ix] # crop and append
44
45     X = torch.tensor(X)
46     Y = torch.tensor(Y)
47     print(X.shape, Y.shape)
48     return X, Y
49
50 import random
51 random.seed(42)
52 random.shuffle(words)
53 n1 = int(0.8*len(words))
54 n2 = int(0.9*len(words))
55
56 Xtr, Ytr = build_dataset(words[:n1])      # 80%
57 Xdev, Ydev = build_dataset(words[n1:n2]) # 10%
58 Xte, Yte = build_dataset(words[n2:])      # 10%
59
60
61 #=====declare utility function=====
62 #用来评估我们手算梯度准确性的参数
63 def cmp(s, dt, t):
64     #传入三个参数。s是一个字符串，用来表示当前参数张量是哪一个layer的
65     #dt是我们手算的梯度
66     #t是Pytorch计算的梯度
67     ex = torch.all(dt == t.grad).item()
68     #torch.all表示逐元素对比，我们手算的和Pytorch官方计算的grad是否相同。返回值是一个布尔张量
69     #.item()表示把布尔tensor的值，也就是布尔值取出。
70     #所以ex就是一个Boolean类型的变量
71     app = torch.allclose(dt, t.grad)
72     #这里判断的就是，是否两个tensor近似相等
73     maxdiff = (dt - t.grad).abs().max().item()
74     #这里是计算，两个tensor之间，各个元素的最大误差的绝对值。
```

```
75     print(f'{s:15s} | exact: {str(ex):5s} | approximate: {str(app):5s} | maxdiff: {maxdiff}')
76     #整行打印出: 参数名、是否精确相等、是否近似相等、最大误差。
77
78
79     #=====初始化参数=====
80     n_embd = 10 # the dimensionality of the character embedding vectors
81     n_hidden = 64 # the number of neurons in the hidden layer of the MLP
82
83     g = torch.Generator().manual_seed(2147483647) # for reproducibility
84     C = torch.randn((vocab_size, n_embd), generator=g)
85     # Layer 1
86     W1 = torch.randn((n_embd * block_size, n_hidden), generator=g) * (5/3)/((n_embd * block_size)**0.5)
87     b1 = torch.randn(n_hidden, generator=g) * 0.1 # using b1 just for fun, it's useless because of
88     # Layer 2
89     W2 = torch.randn((n_hidden, vocab_size), generator=g) * 0.1
90     b2 = torch.randn(vocab_size, generator=g) * 0.1
91     # BatchNorm parameters
92     batch_norm_gain = torch.randn((1, n_hidden))*0.1 + 1.0
93     batch_norm_bias = torch.randn((1, n_hidden))*0.1
94
95     # Note: I am initializing many of these parameters in non-standard ways
96     # because sometimes initializing with e.g. all zeros could mask an incorrect
97     # implementation of the backward pass.
98
99     parameters = [C, W1, b1, W2, b2, batch_norm_gain, batch_norm_bias]
100    print(sum(p.nelement() for p in parameters)) # number of parameters in total
101    for p in parameters:
102        p.requires_grad = True
103
104    batch_size = 32
105    n = batch_size # a shorter variable also, for convenience
106    # construct a minibatch
107    ix = torch.randint(0, Xtr.shape[0], (batch_size,), generator=g)
108    Xb, Yb = Xtr[ix], Ytr[ix] # batch X,Y
109
110
111
```

```

112 #=====训练层=====
113 #forward pass层: 步骤拆分的非常细腻, 方便我们一步步手动backward
114 #embedding层
115 emb = C[Xb] # embed the characters into vectors
116 embcat = emb.view(emb.shape[0], -1) # concatenate the vectors
117
118 # Linear layer 1
119 hpre_batch_norm = embcat @ W1 + b1 # hidden layer pre-activation: 送进BN之前的预激活
120
121 # BatchNorm layer
122 #原本算式: x是BatchNorm层输入的tensor
123 # BatchNorm_output = batch_norm_gain * (x - xmean) / torch.sqrt(xvar + self.eps) + batch_norm_bias
124 batch_norm_meani = 1/n*hpre_batch_norm.sum(0, keepdim=True)#输入tensor内各元素的平均值
125 batch_norm_diff = hpre_batch_norm - batch_norm_meani
126 batch_norm_diff2 = batch_norm_diff**2#为了计算输入tensor的方差, 而存在的中间量
127 batch_norm_var = 1/(n-1)*(batch_norm_diff2).sum(0, keepdim=True) # note: Bessel's correction (dividing by n-1, not n)
128 batch_norm_inverse_std = (batch_norm_var + 1e-5)**-0.5 # inv=inverse(倒数): 这里是 1/sqrt(var+eps), 也就是标准差的倒数
129 batch_norm_raw = batch_norm_diff * batch_norm_inverse_std
130 hpre_batch_norm_out = batch_norm_gain * batch_norm_raw + batch_norm_bias # BN的输出, 也就是tanh的输入
131
132 # Non-linearity
133 h = torch.tanh(hpre_batch_norm_out) # hidden layer
134
135 # Linear layer 2
136 logits = h @ W2 + b2 # output layer
137
138 #拆解交叉熵的计算(The outcome is just the same as the code: F.cross_entropy(logits, Yb))
139 #原式子: 式子: loss = - (1/N) * Σ_i log( exp(logits[i, Yb[i]]) / Σ_j exp(logits[i, j]) )
140 # (0) 巩固一下: logits[i, Yb[i]]是高级查表:
141 # 1.i是一个整数, yb[i]也是一个整数。
142 # 所以整体就是取出样本i, 输入神经网络后, 输出正确的ch, 也就是yb[i]这个数字对应的ch的概率
143
144 # (1) 为什么这里要减去最大值?
145 # 先看这里是怎么减的:
146 # 1.logits的维度是(batch_size, vocab_size), 也就是(32,27)
147 # 2.logits.max(1, keepdim=True).values # dim=1 → 每行一个最大值, 形状 (32,1)
148 # 3.所以是Batch里面的每一个样本, 减去样本内27个分量的最大值

```

```

149
150 # 减去最大值的好处:
151 #     1.防止溢出----logits的值可能很大, 取了exp(logits[i, Yb[i]])可能溢出
152 #     2.比如[100,100.....,100]减去最大值后, 最大的那一分量变成0, 其余都小于0, 就不可能溢出了
153
154 # 并且减去同一个样本内, 各个分量都减去最大值, 不影响最终该样本的Loss:
155 #     1.一个样本i对应的Loss计算式:  $\log(\exp(\text{logits}[i, Yb[i]]) / \sum_j \exp(\text{logits}[i, j]))$ 
156 #     2.那么j, 就表示这个样本i, 对应神经网络输出的长度27的tensor内, 第j个分量
157 #     3.假设减去最大值k, 就会变成:
158 #          $\log(\exp(\text{logits}[i, Yb[i]] - k) / \sum_j \exp(\text{logits}[i, j] - k))$ 
159 #     4.先看分母:  $\sum_j \exp(\text{logits}[i, j] - k)$ :
160 #         若logits[i,j]记为s_j,
161 #         那么分母就是: e的s_1-k次方 + e的s_2-k次方 + ... + e的s_27-k次方
162 #         就等于 (e的s_1次方 + e的s_2次方 + e的s_27次方) / e的k次方
163 #     5.再看分子:  $\exp(\text{logits}[i, Yb[i]] - k)$ 
164 #         等价于: e的t-k次方, 也就是e的t次方 / e的k次方
165 #     6.上下消去了最大值k, 所以总的来说, 该操作对计算没有任何影响
166 logit_maxes = logits.max(1, keepdim=True).values
167 norm_logits = logits - logit_maxes
168 counts = norm_logits.exp()
169 counts_sum = counts.sum(1, keepdim=True) #对每一个样本内的元素进行求和
170 counts_sum_inverse = counts_sum**-1 # inv=inverse: counts_sum的倒数。
171 probs = counts * counts_sum_inverse
172 logprobs = probs.log()
173 loss = -logprobs[range(n), Yb].mean()
174 # (2) 这里的语法: 为什么可以放入一个range(n)来查表?
175 #     1.logprobs 形状是 (n, vocab_size), 也就是 (32, 27)。n就是batch_size
176 #     2.给一个二维张量两个索引的时候, tensor会智能的把他们配对: 效果等价如下。是一种高级索引
177 #     [
178 #         logprobs[0, Yb[0]],
179 #         logprobs[1, Yb[1]],
180 #         ...
181 #         logprobs[n-1, Yb[n-1]]
182 #     ]
183
184 # PyTorch backward pass
185 for p in parameters:

```

```
186 p.grad = None
187 for t in [logprobs, probs, counts, counts_sum, counts_sum_inverse, # afaik there is no cleaner way
188           norm_logits, logit_maxes, logits, h, hpre_batch_norm_out, batch_norm_raw,
189           batch_norm_inverse_std, batch_norm_var, batch_norm_diff2, batch_norm_diff, hpre_batch_norm, batch_norm_mean1,
190           embcat, emb]:
191     t.retain_grad()
192     #表示把所有的中间参与过运算的tensor都保留其grad。方便我们后续对比
193     #本来应该只有w参数们，作为叶子tensor，才会被保留grad
194 loss.backward()
195
196
197
198 #=====练习一：手动实现该神经网络所有的backward=====
199 #前置：矩阵对矩阵求导的实质：
200 # 1.就是澄清一个易混淆的点：Loss是一个矩阵，Logits也是一个矩阵，那么Loss对Logits求偏导是什么意思？
201 # 2.就是对于Loss里面每一个分量a，对Logits里面每一个和a有关的变量b，求偏导。
202 # 3.所以矩阵对矩阵求导，其实是对矩阵内具体的值求导
203
204 #核心：对于一个tensor内的value对象来说：
205 # 对应一个value，也就是tensor内的一个元素而言，
206 # backward等于，接收并且累加它的子value传给它的grad，
207 # 且计算经由自己传导给自己父value们的grad并且传给父元素
208 # 子元素，就是forward pass的过程中，会使用本value的输出作为其输入计算其data的value
209 # 负元素，就是在forward pass中，作为输入计算本value的data的value
210
211 #一、第一步：从Loss，来backward到logits
212 # (1) 先从Loss，backward到logprob:
213 # 1.probs矩阵，是这样的：(Batch_size,vocab_size)。
214 # 表示每一个样本，输入神经网络之后，神经网络根据这个输入，猜测下一个输出的ch的概率分布
215 # 也就是下一个输出是这27种ch中的每一种的概率是多大
216 # 比如probs[1,1]就是，第一个样本，输入神经网络之后，神经网络预测输出下一个ch字符是a的概率
217 #
218 # 2.但是，对于logprob矩阵，每一个样本，都只有一个元素被拿去计算Loss了。一共被拿了Batch_size个
219 # 详见：loss = -logprobs[range(n), Yb].mean()
220 # 所以说，logprob矩阵内，只有一部分元素，和Loss有关。只有这些有关的元素，才会在backward过程被遍历到
221 # 别的元素的grad都默认是0，不会受到任何影响。因为它们的改变并不会导致Loss的改变，自然grad应该是0
222 #
```

```

223 # 3.假设, a, b, c.....z都是和Loss有关的, 那么Loss实际上就是: Loss = - (1.0/Batch_size) * (a+b+c+...+z)
224 # 也就是: Loss = (-1/32) a + (-1/32) * b + ... + (-1/32) * z
225 #
226 # 4.所以我们就找出, logprobs矩阵内, 被选择过的
227 derivative_logprobs = torch.zeros_like(logprobs)
228 derivative_logprobs[range(n), Yb] = -1.0/n
229
230 # (2) 从logprobs, 推回到probs
231 # 1.只有derivative_logprobs[range(n), Yb], 这部分选择的元素, 才是有grad, 才需要向上追溯
232 # 2.那么对于每一个derivative_logprobs[range(n), Yb]选中的元素, 都对应probs[range(n), Yb]选中的元素唯一对应
233 # 3.所以假设a是logprobs的一个有关元素, 在probs中唯一对应的有关元素是b
234 # 所以: a = lnb 。
235 # 那么: da / db = 1.0/b
236 derivative_probs = (1/probs) * derivative_logprobs
237
238
239 # (3) 从probs, 推回到counts和counts_sum_inverse。
240 # 1.其实counts_sum_inverse也是从counts来的
241 # 2.counts是一个(batch_size,vocab_size) 的tensor。每一个元素的含义是, 一个样本内的一个分量的相对频率
242 # 3.我们知道porbs[1,1], 就是第一个样本, 输入神经网络之后, 神经网络预测输出下一个ch字符是a的概率
243 # 4.就是由counts[1,1]乘counts_sum_inverse[0]得来的
244 # 5.counts_sum_inverse[0], 就是counts[0]这个一维tensor内, 各个分量求和, 然后取倒数。
245 # 6.counts_sum_inverse这个矩阵规格是[32,1]。
246 # 7.但是counts和counts_sum_inverse规格不同, 后者只有一列。所以会触发复制。复制出27行变成[32,27]的矩阵
247 # 然后再进行element-wise, 逐元素乘法
248 # 7.所以逐元素表达式: 若probs[i][j] = a ; counts[i][j] = b , counts_sum_inverse[i] = c
249 # 我们有: a = bc
250 # 所以 da / dc = b
251 derivative_counts_sum_inverse = (counts*derivative_probs).sum(1,keepdim=True)
252 #注意: counts*probs是对应元素相乘, 不是矩阵乘法
253 # (4) 为什么这里还需要补一个.sum(1,keepdim=True)?
254 # 1.回忆前向传播的时候:
255 # 如果当前层的一个Value, 被多个下一层的Value使用为输出。那么当backward的时候, 传回的grad会叠加。
256 # 比如:
257 # 当前层有a, 下一层分别是: b = s + a, c = at
258 # 那么dL / da = (dL/db) * (db/da) + (dL/dc) * (dc/da)
259 # 2.所以对于counts*derivative_probs. 这个操作得到的矩阵 K 仍然是(batch_size,vocab_size)大小的

```

```
260 #      probs的第[i]行的所有元素, 都有counts_sum_inverse[0]这个元素作为父元素。
261 #      显然矩阵K的[i][j]元素, 就是probs[i][j]关于父元素counts_sum_inverse[i]的grad。
262 #      所以我们要对k, 沿着行, 跨过列来求和, 也就是把grad都累加到这一行的共有父元素counts_sum_inverse[i]
263 #
264 derivative_counts = derivative_probs* counts_sum_inverse
265 # (5) 这里求counts的derivative。
266 #      对于a=probs[i][j], 一个父元素是b=counts[i][j], 另一个就是c=counts_sum_inverse[i]。
267 #      所以dL / db = (dL / da) * (da / db)
268 #      并且 (da / db) = c
269 #      很显然得到上面式子
270 #
271 # (6) 既然counts_sum_inverse也是从counts来的, 那为什么不直接得到counts? 还要计算counts_sum_inverse?
272 #      因为counts被使用了两次, 它有两个直接子value矩阵, 一个是probs, 一个是counts_sum = counts.sum(1, keepdim=True)
273 #      所以要等它所有的子value矩阵, 把全部grad都返回给它身上, 才可以真正得到dL / d counts
274
275
276
277 # (7) 从counts_sum_inverse得到counts:
278 #      1.先从counts_sum_inverse 得到 counts_sum:
279 #      若  $y = 1/x$ , 那么  $dy / dx = -1 / (x**2)$ 
280 #      2.并且counts_sum_inverse[i][j]的父元素有且只有counts_sum[i][j], 直接逐元素计算
281 derivative_counts_sum = -derivative_counts_sum_inverse * counts_sum_inverse**2
282 #      3.再从counts_sum得到counts: 父子关系如下:
283 #      counts_sum[i] = counts[i][1] + counts[i][2] +... counts[i][27]
284 #      4.很显然, 它要传给每个父元素的grad都是1
285 derivative_counts += torch.ones_like(counts)*derivative_counts_sum
286
287
288 # (8) 从counts到Norm_logits:
289 #      1.counts和norm_logits大小都是(batch_size, vocab_size), 在这个例子就是 (32,27)
290 #      2.对于a = counts[i][j], b = logits[i][j], 我们有  $a = e^b$ 
291 #      3.所以da / db = e的b次方, 也就是a。所以 dL / db = (dL / da) * (da / db)
292 derivative_norm_logits = counts*derivative_counts
293
294
295 # (9) 从norm_logits到Logits:
296 #      1.norm_logits. Logits都是(32,27)。即(batch size, vocab size)。Logits maxes是 (32,1)
```

```

297 # 2.对于a = norm_logits[i][j], b = logits[i][j], c = logits_maxes[i], 我们有a = b - c
298 # 3.所以 a 有两个父元素, 一个是b, 一个是c
299 # 4.并且对于同一行的a, 都有一个共同父元素c, 所以a们对c的grad要累加
300 # 4.先计算logits_maxes的偏导: dL / dc = (dL / da) * (da / dc)
301 derivative_logits_maxes = (-1)*derivative_norm_logits.sum(1,keepdim=True)
302 # 5.再计算Logits的偏导: dL / db = (dL / da) * (da / db)
303 # clone(): 不写的话 derivative_logits 和 derivative_norm_logits 是同一个tensor,
304 # 求导发现d norm_logits / d Logits = 1, 所以L对这两个tensor各个元素的导, 是一一对应相等的。
305 # 但是直接用 = 号会有bug。因为这个本质是赋值操作, 也就是添加指针指向同一个对象, 所以需要clone一份这个对象
306 derivative_logits = derivative_norm_logits.clone()
307
308 # (10) 我们需要从Logits_max计算Logits的grad:
309 # 1.我们有a = Logits_max[i], 和b_j = Logits[i][j], 并且j in range of (0,batch_size-1)
310 # 2.a = max (b_j) -----我不知道这个函数如何求导
311 # 3.呃呃呃, Andrej说其实很简单, 假设b就是Logits[i]这一行的最大值。那么其实就是 a = b, 传递的grad就是1
312 # 4.并且值得注意的是, a的父元素只有b这一个, grad仅仅只会传递给这一行的最大值
313 # 所以采用一位热位编码, 只在每一行的被选取作为最大值的位置, 放1, 再乘上层传下来的grad矩阵, 就能限制grad只传给最大值了
314 derivative_logits += F.one_hot(logits.max(1).indices, num_classes=logits.shape[1]) * derivative_logits_maxes
315
316
317 #=====第二层线性层=====
318 #(1)线性层实质: Logits = H @ W2 + B2
319 # 1.各矩阵规格:
320 # Logits为 (Batch_size, vocab_size)
321 # H 为 (Batch_size, n_hidden)
322 # W2为 (n_hidden, vocab_size)
323 # B2为 (vocab_size, 1), 但是由于广播机制, 相当于里面的每一个元素都被重复利用了
324 #
325 # 假设我们有C = A@B + E计算, 我们希望求C对A的导
326 # [ 矩阵计算展开 ]
327
328 # [ a11 a12 ] [ b11 b12 ] [ e1 e2 ] [ c11 c12 ]
329 # [ a21 a22 ] @ [ b21 b22 ] + [ e1 e2 ] = [ c21 c22 ]
330
331 # 展开得到:
332 # c11 = a11*b11 + a12*b21 + e1
333 # c12 = a11*b12 + a12*b22 + e2

```

```

334 #     c21 = a21*b11 + a22*b21 + e1
335 #     c22 = a21*b12 + a22*b22 + e2
336
337 #     [ 梯度反向传播求导 ]
338 #     dL/da11 = (dL/dc11)*b11 + (dL/dc12)*b12
339 #     dL/da12 = (dL/dc11)*b21 + (dL/dc12)*b22
340 #     dL/da21 = (dL/dc21)*b11 + (dL/dc22)*b12
341 #     dL/da22 = (dL/dc21)*b21 + (dL/dc22)*b22
342
343 #     [ 写成矩阵形式 (最终结论) ]
344 #     [ dL/da11  dL/da12 ] [ dL/dc11  dL/dc12 ] [ b11  b21 ]
345 #     [ dL/da21  dL/da22 ] = [ dL/dc21  dL/dc22 ] @ [ b12  b22 ]
346 #     即: dL/dA = dL/dC @ B^T
347 #     这里的n_hidden参数是64
348 # (2) Logits = H @ W2 + B2, W2的规格是(n_hidden, vocab_size)
349 #     所以 dL/dH = dL/dLogits @ W2.T, 必须转置, 否则矩阵乘法维度对不上
350 derivative_h = derivative_logits @ W2.T
351 derivative_W2 = h.T @ derivative_logits
352 #     注意: b2的规格是(vocab_size, ), 在前向里被广播到整个batch, 所以要求的是
353 #     “上层梯度” derivative_logits 的跨样本和, 不是前向的 logits.sum(0) (那是数据, 不是梯度)
354 derivative_B2 = derivative_logits.sum(0)
355 #这里我忘了B2的规格, 应该是 (1, n_hidden), 表示对于一个样本的所有激励值加上bias。
356 #所以backward的时候, 需要跨样本, 沿着同一个特征向量分量求和
357
358
359 #=====激活函数层=====
360 derivative_hpreact = (1-h**2)*derivative_h
361
362 #=====batch_norm层=====
363 # (1) 对这个算式backward: hpre_batch_norm_out = batch_norm_gain * batch_norm_raw + batch_norm_bias
364 #     注意: 传进本层的上层梯度是 derivative_hpreact (tanh那一层已经算好了), 不是 derivative_h
365 derivative_batch_norm_gain = (derivative_hpreact * batch_norm_raw).sum(0,keepdim=True)
366 #这里我忘了, gain和bias都是 (1, n_hidden), 表示对于一个样本的所有激励值加上各自bias, 并且乘各自的gain参数
367 derivative_batch_norm_raw = (batch_norm_gain * derivative_hpreact)
368 derivative_batch_norm_bias = (derivative_hpreact).sum(0,keepdim=True)
369
370 # (2) 目标: batch norm raw = batch norm diff * batch norm inverse std

```

```

371 # 1.规格: batch_norm_raw是(batch_size,n_hidden), 表示每一个样本的每一个元素, 都经过了初步normalize
372 #      batch_norm_diff同上。表示每一个样本的每一个向量, 都减去了跨样本的该向量平均值, 得到的diff矩阵
373 #      batch_norm_inverse_std是 (1, n_hidden) , 表示每一个样本的每一个向量, 都跨样本求出了std, 然后倒置
374 derivative_batch_norm_diff = derivative_batch_norm_raw * batch_norm_inverse_std
375 # 2.这里是“先逐元素相乘、再跨样本求和”: dL/d inv_std_j =  $\sum_i (dL/d \text{raw}_{ij}) * \text{diff}_{ij}$ 
376 #      不能写成 derivative_batch_norm_raw * batch_norm_diff.sum(0,keepdim=True)
377 derivative_batch_norm_inverse_std = (derivative_batch_norm_raw * batch_norm_diff).sum(0,keepdim=True)
378 # 3.inverse_std = (var + eps)**-0.5, 所以 d inv_std / d var = -0.5*(var+eps)**-1.5
379 #      这里用的是 batch_norm_var (不是 batch_norm_inverse_std) , 而且这个量其实是 dL/dvar
380 derivative_batch_norm_var = (-0.5*(batch_norm_var+1e-5)**(-1.5)) * derivative_batch_norm_inverse_std
381
382 # (3) 目标: batch_norm_var = 1/(n-1)*(batch_norm_diff2).sum(0, keepdim=True)。n就是Batch_size
383 #      batch_norm_diff2 = batch_norm_diff**2
384 #
385 # 1.假设var是1*2, 那么显然diff2是2*2。那么 var11 = 1/ (n-1) * (diff2_11 + diff2_21)
386 #      所以dvar / d diff2 = 一个2*2的矩阵, 每一个元素都是1/ (n-1)
387 derivative_batch_norm_diff2 = torch.ones_like(batch_norm_diff2)*(1/(n-1)) * derivative_batch_norm_var
388 # 2.diff2 = diff**2, 所以 d diff2 / d diff = 2*diff, 用的是 batch_norm_diff (不是 batch_norm_diff2)
389 derivative_batch_norm_diff += 2*batch_norm_diff*derivative_batch_norm_diff2
390
391 # (4) 目标: 先batch_norm_diff = hpre_batch_norm - batch_norm_meani,这里meani涉及了广播机制
392 #      再batch_norm_meani = 1/n*hpre_batch_norm.sum(0, keepdim=True)#输入tensor内, 跨样本的分量的平均值
393 # 1.clone()是必须的: 若写成 derivative_hpre = derivative_batch_norm_diff,
394 #      两个名字会指向同一个tensor, 下面的 += 会就地改掉 derivative_batch_norm_diff
395 derivative_hpre = derivative_batch_norm_diff.clone()
396 # 2.diff = hpre - meani, diff对meani的导数是-1, 所以要跨样本累加
397 derivative_batch_norm_meani = (-derivative_batch_norm_diff).sum(0,keepdim=True)
398 # 3.meani = 1/n * hpre.sum(0), 每个样本的hpre都要分到 (1/n)*dL/dmean
399 derivative_hpre += torch.ones_like(hpre_batch_norm)*(1.0/n)*derivative_batch_norm_meani
400
401
402 #=====第一线性层=====
403 # (1) 线性层: hpre_batch_norm = embcat @ W1 + b1
404 # 1.各矩阵规格:
405 #      hpre_batch_norm 为 (Batch_size, n_hidden) = (32, 64)
406 #      embcat          为 (Batch_size, n_embd*block_size) = (32, 30)
407 #      W1              为 (n_embd*block_size, n_hidden) = (30, 64)

```

```

408 #      b1          为 (n_hidden,) = (64,)
409 #      2.矩阵求导的结论和上面第二线性层一模一样，只是字母换了：
410 #      若  $C = A @ B$ ，则  $dL/dA = dL/dC @ B.T$ ,  $dL/dB = A.T @ dL/dC$ 
411 derivative_embcat = derivative_hpre @ W1.T
412 derivative_W1 = embcat.T @ derivative_hpre
413 #      3.b1也是被广播的（一个(64,)的向量被加到32个样本上），所以要把上层梯度沿着样本维度sum掉
414 derivative_b1 = derivative_hpre.sum(0)
415 # (2) 顺带说一个有趣的事实：b1其实是个“废物参数”
416 #      因为hpre_batch_norm马上就要送进BatchNorm，而BN第一步就是减去跨样本均值batch_norm_meani。
417 #      给每个分量加上b1，只是让整列整体平移，这个平移量在“减去均值”的瞬间就被彻底消掉了，
418 #      所以b1对最终Loss没有任何影响（它的grad虽然能算出来，但学不到任何东西）。
419
420
421 #=====view层（拼接）=====
422 # (1) embcat = emb.view(Batch_size, -1)，这是一个纯粹的“形状变换”：
423 #      1.它不改变内存里元素的存放顺序，只是把(32, 3, 10)重新解释成(32, 30)
424 #      2.所以backward就是它的逆操作：把(32, 30)再view回(32, 3, 10)
425 #      3.两种形状同一位置的元素一一对应，所以既不需要sum，也不需要复制
426 derivative_emb = derivative_embcat.view(emb.shape)
427
428
429 #=====embedding层=====
430 # (1) emb = C[Xb]，这一步是“查表”：
431 #      1.C的规格是(vocab_size, n_embd)=(27, 10)，Xb的规格是(Batch_size, block_size)=(32, 3)
432 #      2.输出的emb[i][j]就等于C的第Xb[i][j]行，也就是把C里的某一行“复制”到emb里
433 #      3.所以C的每一行都会被复制很多次（同一个字符在同一个batch里可能出现多次）
434 #      4.于是backward的时候，C的每一行要把所有“复制过它的位置”传回来的grad累加起来
435 # (2) 所以这里就直接用一个双重循环，把“累加”这件事老老实实写出来：
436 #      注意要用zeros_like先开一块新内存，不能直接在C.grad上累加，
437 #      因为我们要自己算一份梯度，去和PyTorch算的C.grad做对比
438 derivative_C = torch.zeros_like(C)
439 for i in range(n):
440     for j in range(block_size):
441         character_index = Xb[i][j] # 这个位置上查表用的字符编号
442         derivative_C[character_index] += derivative_emb[i][j]
443         #这里emb是一个三维tensor，所以每一次加回去给C，都是加一个维度的tensor。
444         #也就是每一次加给C，都是加上一个字符对应的10维向量

```

```
445
446
447
448 #=====练习二：用cmp验证我们手算的梯度=====
449 # (1) 使用说明：cmp(s, dt, t)会把我们手算的dt和 t.grad (PyTorch算的) 作对比
450 # 1.所以传进来的t必须是前面retain_grad()过的中间张量，或者是requires_grad的参数
451 # 2.不要强求 exact=True: 浮点数的加减乘除结合顺序不同，结果就会有1e-9左右的差异，
452 # 只要 approximate=True、maxdiff 在1e-8量级，就说明我们的推导是正确的
453 # (2) 从最后一层往回，逐个对比（包括参数和所有中间张量）：
454 cmp('C', derivative_C, C)
455 cmp('emb', derivative_emb, emb)
456 cmp('embcats', derivative_embcats, embcats)
457 cmp('W1', derivative_W1, W1)
458 cmp('b1', derivative_b1, b1)
459 cmp('hpre_batch_norm', derivative_hpre, hpre_batch_norm)
460 cmp('batch_norm_mean1', derivative_batch_norm_mean1, batch_norm_mean1)
461 cmp('batch_norm_diff', derivative_batch_norm_diff, batch_norm_diff)
462 cmp('batch_norm_diff2', derivative_batch_norm_diff2, batch_norm_diff2)
463 cmp('batch_norm_var', derivative_batch_norm_var, batch_norm_var)
464 cmp('batch_norm_inverse_std', derivative_batch_norm_inverse_std, batch_norm_inverse_std)
465 cmp('batch_norm_raw', derivative_batch_norm_raw, batch_norm_raw)
466 cmp('batch_norm_gain', derivative_batch_norm_gain, batch_norm_gain)
467 cmp('batch_norm_bias', derivative_batch_norm_bias, batch_norm_bias)
468 cmp('hpre_batch_norm_out', derivative_hpreact, hpre_batch_norm_out)
469 cmp('h', derivative_h, h)
470 cmp('W2', derivative_W2, W2)
471 cmp('b2', derivative_B2, b2)
472 cmp('logits', derivative_logits, logits)
473 cmp('logit_maxes', derivative_logits_maxes, logit_maxes)
474 cmp('norm_logits', derivative_norm_logits, norm_logits)
475 cmp('counts', derivative_counts, counts)
476 cmp('counts_sum', derivative_counts_sum, counts_sum)
477 cmp('counts_sum_inverse', derivative_counts_sum_inverse, counts_sum_inverse)
478 cmp('probs', derivative_probs, probs)
479 cmp('logprobs', derivative_logprobs, logprobs)
480
481
```

